Tree

沈威宇

November 9, 2026

1 Tree

I Tree

i Tree

A tree is an abstract data type of a set of nodes. A relationship between two nodes i, j is defined with

- i is called a child of j and
- j is called a parent of i; and
- a node can have at most one parent.

If not empty, a tree is recursively defined as a distinguished node r called root and zero or more nonempty trees called subtrees, where the root of each substree is a child of r.

ii Terminology

- Siblings: Nodes sharing the same parent are called siblings. Children of a node are defined with a left to right order.
- Leaf: A node with no children is called a leaf, external node, or exterior node.
- Internal node: A node that is not a leaf is called an internal node or interior node.
- Depth: The number of edges from the root to a node is called the depth of the node, while the root has depth 0.
- Height: The height of a leaf is defined as 0. The height of an internal node is defined as the maximum height of its children plus 1. The maximum depth of the nodes, i.e., the height of its root, is called the height or depth of the tree. The height or depth of an empty tree is defined as -1. Some define heights of nodes and trees as the definition here plus 1, which is not used here.
- Ancestors: The ancestors of a node is recursively defined as itself for root and itself and the ancestors of its parent for non-root nodes.
- Descendants: The descendants of a node is recursively defined as itself for leaves and itself and the descendants of its children for internal nodes.
- Level: Nodes of the same depth are called a level. Levels are ordered by depth. The left-to-right order of nodes in a level is defined recursively to, if they are siblings, follow that of siblings; otherwise follow that of their parents.
- Subtree: A tree consisting of all descendants of a node is called a subtree. The left-to-right order of subtrees whose roots are of the same depth are defined as

that of their roots. A subtree whose root is a child of a node is called a subtree of the node.

iii Tree representation

A representation of tree where each pair of parent and child is connected with a edge.

iv Left child, right sibling representation

A representation of tree where each internal node and its leftmost child is connected with a edge, and each sibling that is not the leftmost one is called to the sibling at its left.

v Skewed

A tree whose every level contains only one node is called skewed.

II n-ary tree

i n-ary tree

A tree in which no node can have more than n children is called an n-ary tree. A child is indexed into one and only one integer in $[1, n]$ even if its parent has less than n children.

ii Full, proper, or plane

An n-ary tree where each internal node has exactly n children is called full, proper, or plane.

iii Filled

A level of depth $k \in \mathbb{N}$ of an n-ary tree is called filled iff each node in the level of depth $k - 1$ has exactly n children. The level of depth 0 is called filled iff the tree is nonempty.

iv Complete

An n-ary tree where every level, except possibly the last, is filled and whose nodes in the last level are as far left as possible, is called complete.

A complete n-ary tree is indexed from top to bottom and then from left to right, with the root as node 1.

v Perfect

A complete n-ary tree where all leaves are of the same depth is called perfect.

A perfect n-ary tree with height k has $\sum_{j=0}^{k} n^j$ nodes.

III Binary tree

i Binary tree

In a binary tree, the two possible children of a node are called left child and right child. A child is either left or right but not both even if it has no sibling. The left and right subtrees of a node x are denoted left(x) and right(x) respectively.

ii Complete binary tree

For a complete binary tree with n nodes:

- Parent of node i is node $\left\lfloor \frac{i}{2} \right\rfloor$ for all integers $i > 1$.
- The left child of node i is $2i + 1$ for all integers $i \leq \frac{n-1}{2}$.
- The right child of node i is $2i$ for all integers $i \leq \frac{n}{2}$.

iii Perfect binary tree

A perfect binary tree with height k has $2^{k+1} - 1$ nodes.

iv Balanced tree

A type of binary tree with height $O(\log n)$ where n is the number of nodes is called balanced.

v Balance factor

The height of the left subtree minus the height of the right subtree of a node in a binary tree is called the balance factor of the node.

vi Left-skewed and right-skewed

A binary tree whose every internal node has left child and does not have right child is called left-skewed.

A binary tree whose every internal node has right child and does not have left child is called right-skewed.

IV Tree traversal

They are defined recursively. Suppose the root is r.

i Preorder traversal

First visit r, and then visit the subtrees of r from left to right.

ii Postorder traversal

Visit the subtrees of r from left to right, and finally visit r.

iii Inorder traversal (binary tree)

First visit the left subtree of r, and then visit r, and then visit the right subtree of r.

iv Full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals

A full binary tree with distinct values can be uniquely determined by its preorder and postorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a full binary tree from:
 *   P - preorder array (size num)
 *   Q - postorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *Q, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  Node *root = createNode(P[0]);
  int leftRootVal = P[1];
  int k = 0;
  while (Q[k] != leftRootVal)
    k++;
  int leftSize = k + 1;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, Q, leftSize);
  root->right = construct(P + 1 + leftSize, Q + leftSize, rightSize);
  return root;
```

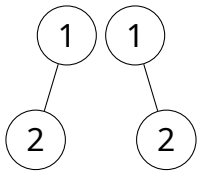
```
}
```

□

v Binary tree with distinct values cannot be uniquely determined by its preorder and postorder traversals

The statement that a binary tree with distinct values can be uniquely determined by its preorder and postorder traversals is false.

Proof. No. A counterexample is the following two binary trees



Both trees have preorder traversal 1, 2, and postorder traversal 2, 1, but they are different. □

vi Binary tree with distinct values can be uniquely determined by its preorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its preorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
 *   P - preorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *P, int *I, int num) {
  if (num <= 0)
```

```
    return NULL;
  if (num == 1)
    return createNode(P[0]);
  int rootVal = P[0];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
  int rightSize = num - 1 - leftSize;
  root->left = construct(P + 1, I, leftSize);
  root->right = construct(P + 1 + leftSize, I + leftSize + 1, rightSize);
  return root;
}
```

□

vii Binary tree with distinct values can be uniquely determined by its postorder and inorder traversals

A binary tree with distinct values can be uniquely determined by its postorder and inorder traversals.

Proof. The following C code provides an algorithm to reconstruct the tree recursively:

```c
#include <stdlib.h>

typedef struct Node {
  int data;
  struct Node *left;
  struct Node *right;
} Node;

Node *createNode(int data) {
  Node *n = (Node *)malloc(sizeof(Node));
  n->data = data;
  n->left = NULL;
  n->right = NULL;
  return n;
}

/*
 * Reconstruct a binary tree from:
 *   Q - postorder array (size num)
 *   I - inorder array (size num)
 *   distinct values assumed
 */
Node *construct(int *Q, int *I, int num) {
  if (num <= 0)
    return NULL;
  if (num == 1)
    return createNode(Q[num - 1]);
  int rootVal = Q[num - 1];
  Node *root = createNode(rootVal);
  int k = 0;
  while (I[k] != rootVal)
    k++;
  int leftSize = k;
```

```
  int rightSize = num - 1 - k;
  root->left = construct(I, Q, leftSize);
  root->right = construct(I + k + 1, Q + leftSize, rightSize);
  return root;
}
```

□

V Binary search tree (BST)

i Binary search tree (BST)

A binary search tree stores keys in the nodes of a binary tree so that searching, insertion and deletion can be done efficiently. For every node x, all the keys in its left subtree are smaller than x and all keys in its right subtree are larger than x. Thus, the values are distinct.

ii Searching

Suppose we are searching for x and the root is r.

- If $x = r$, then we are done.
- If $x < r$, then we search in the left subtree.
- If $x > r$, then we search in the right subtree.

The worst-case and amortized time complexities are $O(\text{height of tree})$.

iii Finding minimum and maximum

To find the node containing the smallest key, start at root and go to the left child as long as there is one, and the leaf we stop at is the smallest key. Similarly for find the largest key.

The worst-case and amortized time complexities are $O(\text{height of tree})$.

iv Insertion

Suppose we want to insert x. First search for it, if we found it, do noting (or update something); otherwise, if we stop at y,

- if $x > y$, insert x as the right child of y if $x > y$, and
- if $x < y$, insert x as the left child of y.

The worst-case and amortized time complexities are $O(\text{height of tree})$.

v Deletion

Suppose we want to delete x. First search for it, if we don't found it, do noting (or update something); otherwise,

- if it is a leaf, delete it,
- if it has one child y, adjust y as a child of the parent of x of the same position as x and delete x, and
- if it has two children, choose either of the policies:
  - inorder successor: replace the value of x with the value of the minimum element y in the right subtree of x, i.e. inorder successor, i.e., the leftmost descendant of the right child of x, which has no left child but may have right child, and
    - remove node y if it has no child, and
    - replace node y with its right child if it has one;
  - inorder predecessor: replace the value of x with the value of the maximum element y in the left subtree of x, i.e. inorder predecessor, i.e., the rightmost descendant of the left child of x, which has no right child but may have left child, and
    - remove node y if it has no child, and
    - replace node y with its left child if it has one;

The worst-case and amortized time complexities are $O(\text{height of tree})$.

vi Rotation

A (simple) rotation on an edge between a node x and its parent p is a tree operation on BST and has the following two symmetric variants:

- Right rotation: This is done when x is a left child of p. We perform the following assignments:

$$\text{left}(p) = \text{right}(x)$$

$$\text{right}(x) = p$$

- Left rotation: This is done when x is a right child of p. We perform the following assignments:

$$\text{right}(p) = \text{left}(x)$$

$$\text{left}(x) = p$$

vii Uniqueness given preorder traversal

A BST is uniquely determined by its preorder traversal.

Proof. A BST with one node is obviously unique. The first element must be root, the second element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

viii Uniqueness given postorder traversal

A BST is uniquely determined by its postorder traversal.

Proof. A BST with one node is obviously unique. The last element must be root, the first element to the last element smaller than root must be the left subtree, and the rest must be the right subtree. Recursively partition into the subtrees. We uniquely obtained the BST. □

ix Binary tree is BST iff inorder traversal is in ascending order

A binary tree with distinct values is a binary search tree iff its inorder traversal is sorted in ascending order.

Proof. Only if: Prove by mathematical induction. The inorder traversal of a BST with one node is sorted in ascending order. Assume the inorder traversal of any BST with fewer than n nodes of distinct values is sorted in ascending order. Consider any BST with n nodes, the inorder traversal visits the left subtree, which is smaller than root and itself sorted in ascending order since it's a BST with $n-1$ nodes, and then root, and then the right subtree, which is larger than root and itself sorted in ascending order since it's a BST with $n-1$ nodes. Thus, we conclude that the inorder traversal of any BST with n nodes is sorted in ascending order. By mathematical induction, the statement holds.

If: Prove by mathematical induction. A binary tree with one node is a BST. Assume that any binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order is a BST. Consider any binary tree with n nodes of distinct values and root r whose inorder traversal is sorted in ascending order. The subsequence before r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are smaller than r. The subsequence after r is a binary tree with fewer than n nodes of distinct values whose inorder traversal is sorted in ascending order whose values of nodes are larger than r. Since the left subtree is a BST and smaller than root and the right subtree is a BST and larger than root, we conclude that any binary tree with n nodes of distinct values whose inorder traversal is sorted in ascending order is a BST. By mathematical induction, the statement holds. □

x Number of distinct BST shapes

The number T_n of distinct shapes of BSTs with n nodes is:

$$T_0 = 1$$

$$T_n = \sum_{i=0}^{n-1} T_i T_{n-1-i}, \quad n \in \mathbb{N}$$

$$T_n = \frac{1}{(n+1)} \binom{2n}{n}, \quad n \in \mathbb{N}_0$$

Proof. Obviously

$$T_0 = T_1 = 1$$

Let the values be

$$k_1 < k_2 < \ldots < k_n$$

Suppose we choose k_i as root, where $1 \leq i \leq n$. The left subtree contains $k_1, \ldots, k_{i-1}$, so it has $i-1$ nodes and T_{i-1} possible combinations. The right subtree contains $k_{i+1}, \ldots, k_n$, so it has $n-1$ nodes and T_{n-i} possible combinations.

Summing over all choices gives

$$T_n = \sum_{i=1}^{n} T_{i-1} T_{n-i} = \sum_{i=0}^{n-1} T_i T_{n-1-i}$$

Using generating function:

$$C(x) = \sum_{n=0}^{\infty} T_n x^n$$

$$C(x) = 1 + xC(x)^2$$

$$C(x) = \frac{1 \pm \sqrt{1-4x}}{2x}$$

$$C(0) = \mp \frac{1}{\sqrt{1-4x}} = T(0) = 1$$

$$C(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

$$
\begin{aligned}
\sqrt{1-4x} &= \sum_{k=0}^{\infty} \binom{0.5}{k} (-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{\prod_{i=0}^{k-1}(0.5-i)}{k!}(-4x)^k \\
&= 1 + \sum_{k=1}^{\infty} \frac{2^k \prod_{i=0}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{k-1}(2i-1)}{k!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k \prod_{i=1}^{2k-2} i}{k! \prod_{i=1}^{k-1} 2i}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2^k (2k-2)!}{k! 2^{k-1}(k-1)!}x^k \\
&= 1 + \sum_{k=1}^{\infty} -\frac{2(2k-2)!}{k!(k-1)!}x^k
\end{aligned}
$$

$$
\begin{aligned}
C(x) &= \frac{1-\sqrt{1-4x}}{2x} \\
&= \sum_{k=1}^{\infty} \frac{(2k-2)!}{k!(k-1)!}x^{k-1} \\
&= \sum_{n=0}^{\infty} \frac{(2n)!}{(n+1)!n!}x^n \\
&= \sum_{n=0}^{\infty} \frac{1}{(n+1)}\binom{2n}{n}x^n
\end{aligned}
$$

$$
T_n = \frac{1}{(n+1)}\binom{2n}{n}, \quad n \in \mathbb{N}_0
$$

□

VI AVL tree (Adelson-Velsky and Landis tree)

i AVL tree (Adelson-Velsky and Landis tree)

An AVL tree is a self-balancing binary search tree (BST) that keeps info of height, invented by Georgy Adelson-Velsky and Evgenii Landis. An AVL tree needs to comply with the property that for every node with two children in the tree, the height of its left and right subtrees differ by at most 1.

Some define heights of nodes and AVL trees as the definition used here plus 1, which is not used here.

ii Relation of height and numbers of nodes

Let the minimum number of nodes in an AVL tree of height h be N_h. Since the minimum numbers of subtrees height of the root of an AVL tree of height h are $h-1$ and $h-2$, we have:

$$N_{-1} = 0$$

$$N_0 = 1$$

$$N_h = N_{h-1} + N_{h-2} + 1, \quad h \in \mathbb{N}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1, \quad h+1 \in \mathbb{N}_0$$

Proof. Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2 + \sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{7+3\sqrt{5}}{2} = \varphi^4 = 3\varphi + 2$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$\frac{3-\sqrt{5}}{2} = \frac{1}{\varphi^2} = 1 - \frac{1}{\varphi} = 2 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$N_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$N_0 = a + b + c = 1$$

$$N_1 = a\varphi - \frac{b}{\varphi} + c = 0 + 1 + 1 = 2$$

$$N_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 1 + 2 + 1 = 4$$

$$N_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 4$$

$$N_2 - N_1 = a + b = 4 - 2 = 2$$

$$c = N_0 - (a + b) = 1 - 2 = -1$$

$$N_1 - c = a\varphi - \frac{2-a}{\varphi} = 2 + 1 = 3$$

$$a\left(\varphi^2 + 1\right) = 3\varphi + 2$$

$$a = \frac{3\varphi + 2}{\varphi + 2} = \frac{\varphi^4}{\varphi + 2}$$

$$b = 2 - a = \frac{2-\varphi}{\varphi+2} = \frac{1}{\varphi^2(\varphi+2)}$$

$$N_h = \frac{1}{\varphi+2}\left(\varphi^{h+4} + \left(-\frac{1}{\varphi}\right)^{h+2}\right) - 1$$

$$\frac{1}{\varphi+2} = \frac{2}{5+\sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$N_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+3} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+3}\right) - 1$$

□

So

$$N_h = \Theta(\left(\frac{1+\sqrt{5}}{2}\right)^h)$$

$$h = \log_{\frac{1+\sqrt{5}}{2}} N_h + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Let the maximum number of nodes in an AVL tree of height h be M_h. Since the maximum numbers of subtrees height of the root of an AVL tree of height h are $h-1$, we have:

$$M_{-1} = 0$$

$$M_h = 2M_{h-1} + 1, \quad h \in \mathbb{N}_0$$

$$M_h = 2^{h+1} - 1, \quad h + 1 \in \mathbb{N}_0$$

So

$$M_h \in \Theta(\log_2 M_h)$$

$$h = \log_2 M_h + O(1)$$

Let the number of nodes be n.

$$N_h \le n \le M_h$$

$$\log_2 n + O(1) \le h \le \log_{\frac{1+\sqrt{5}}{2}} n + O(1) \approx 1.44 \log_2 N_h + O(1)$$

Thus

$$h \in \Theta(\log n)$$

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in AVL tree is the same as in BST.

The worst-case and amortized time complexities are $O(\text{height of tree}) = O(\log \text{number of nodes})$.

iv Rotation

When insertion/deletion, the AVL property may be violated, called unbalanced. Since an insertion/deletion involves adding/deleting a single node, this can only increase/decrease the height of some subtree by 1. Thus, if the AVL property is violated at a node x, it means that the heights of left and right subtrees of it differ by exactly 2. We need to transform the tree to restore the AVL property, called rebalancing, using a single/simple rotation or a double rotation, each has two symmetric variants:

- LL case / right single rotation: Let the left child of x be y. The LL case is where the left subtree of y has height $h+1$, while the right subtree of y and the right subtree of x have height h. We perform a right rotation on the edge between y and x, i.e.,

$$\text{left}(x) = \text{right}(y)$$

$$\text{right}(y) = x$$

- RR case / left single rotation: Let the right child of x be y. The RR case is where the right tree of y has height $h+1$, while the left subtree of y and the left subtree of x have height h. We perform a left rotation on the edge between y and x, i.e.,

$$\text{right}(x) = \text{left}(y)$$

$$\text{left}(y) = x$$

- LR case / left-then-right double rotation: Let the left child of x be y and the right child of y be z. The LR case is where the right subtree of y has height $h+1$, while the left subtree of y and the right subtree of x have height h. We perform a left rotation on the edge between z and y and then a right rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{right}(y) = \text{left}(z)$$

$$\text{left}(z) = y$$

$$\text{left}(x) = \text{right}(z)$$

$$\text{right}(z) = x$$

- RL case / right-then-left double rotation: Let the right child of x be y and the left child of y be z. The RL case is where the left subtree of y has height $h+1$, while the right subtree of y and the left subtree of x have height h. We perform a right rotation on the edge between z and y and then a left rotation on the edge between z, now at the original position of y, and x, i.e.,

$$\text{left}(y) = \text{right}(z)$$

$$\text{right}(z) = y$$

$$\text{right}(x) = \text{left}(z)$$

$$\text{left}(z) = x$$

v Insertion

Insertion in AVL tree involves first doing insertion as with a normal BST, and then tracing the path from the new leaf to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then break the loop since no second violation may exist.

Proof. Let x be the first node encountered when traversing upward from the newly inserted node toward the root that violates the AVL property. Since an insertion involves adding a single node, this can only increase the height of some subtree by 1. Thus, the heights of the left and right subtrees differ by exactly 2. For any of the four cases (LL, RR, LR, RL), since the inserted node must be in the subtree of height $h+1$ of y, the height of the subtree whose root is x has height $h+2$ both before insertion and after rotation. Thus, the fact that the AVL property is satisfied for all ancestors of x before insertion implies that the AVL property is satisfied for all ancestors of x after rotation. □

The worst-case and amortized time complexities are $O(\text{height of tree}) = O(\log \text{number of nodes})$.

vi Deletion

Deletion in AVL tree involves first doing deletion the same as in BST, and then tracing the path from

- the original parent of the deleted node if the deleted node had zero or one child, and
- the original parent of the inorder successor or inorder predecessor that was moved if the deleted node had two children,

to the root: For each node x encountered, check if heights of its left and right subtrees differ by at most 1,

- if yes, proceed to the parent of x;
- if not, we say the AVL property is violated at x and perform rotations to restore it, and then proceed to the parent of x since violation may propagate upwards.

The worst-case and amortized time complexities are $O(\text{height of tree}) = O(\log \text{number of nodes})$.

vii Number of distinct AVL tree shapes

The number $A_{h,n}$ of distinct shapes of AVL trees with n nodes and height h is:

$$A_{-1,0} = 1$$

$$A_{-1,n} = 0, \quad n \neq 0$$

$$A_{0,1} = 1$$

$$A_{0,n} = 0, \quad n \neq 1$$

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}, \quad h \in \mathbb{N}, n \in \mathbb{N}_0$$

Proof. The root uses one node, leaving $n-1$ nodes for the two subtrees. Since the whole tree has height h, the two trees must either have heights $h-1, h-2$ or both have height $h-1$. Thus,

$$A_{h,n} = \sum_{i=0}^{n-1} A_{h-1,i}A_{h-1,n-1-i} + A_{h-1,i}A_{h-2,n-1-i} + A_{h-2,i}A_{h-1,n-1-i}$$

□

viii Number of distinct AVL tree shapes with minimum number of nodes given height

The number B_h of distinct shapes of AVL trees with height h and minimum number of nodes is:

$$B_{-1} = 1$$

$$B_0 = 1$$

$$B_h = 2B_{h-1}B_{h-2}, \quad h \in \mathbb{N}$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right) - 1}, \quad h+1 \in \mathbb{N}_0$$

Proof. To minimize the number of nodes, the two subtrees of root must be the AVL tree with height $h-1, h-2$ and minimum number of nodes, which have B_{h-1}, B_{h-2} distinct shapes respectively. There are two possible arrangements:

- Left subtree has height $h-1$, and right subtree has height $h-2$.
- Left subtree has height $h-2$, and right subtree has height $h-1$.

Thus,

$$B_h = 2B_{h-1}B_{h-2}$$

Let

$$C_h = \log_2 B_h$$

$$C_h = C_{h-1} + C_{h-2} + 1$$

Use characteristic equation:

$$x^2 - x - 1 = 0$$

Let

$$\varphi = \frac{1+\sqrt{5}}{2}$$

Note that

$$\frac{3+\sqrt{5}}{2} = \varphi^2 = \varphi + 1$$

$$2 + \sqrt{5} = \varphi^3 = 2\varphi + 1$$

$$\frac{1-\sqrt{5}}{2} = -\frac{1}{\varphi} = 1 - \varphi$$

$$x = \frac{1 \pm \sqrt{5}}{2} = \varphi, -\frac{1}{\varphi}$$

$$C_h = a\varphi^h + b\left(-\frac{1}{\varphi}\right)^h + c$$

$$C_0 = a + b + c = \log_2 1 = 0$$

$$C_1 = a\varphi - \frac{b}{\varphi} + c = \log_2(2 * 1 * 1) = 1$$

$$C_2 = a\varphi^2 + \frac{b}{\varphi^2} + c = 0 + 1 + 1 = 2$$

$$C_2 = a(\varphi + 1) - b\left(\frac{1}{\varphi} - 1\right) + c = 2$$

$$C_2 - C_1 = a + b = 2 - 1 = 1$$

$$c = C_0 - (a + b) = 0 - 1 = -1$$

$$C_1 - c = a\varphi - \frac{1-a}{\varphi} = 1 + 1 = 2$$

$$a\left(\varphi^2 + 1\right) = 2\varphi + 1$$

$$a = \frac{2\varphi + 1}{\varphi + 2} = \frac{\varphi^3}{\varphi + 2}$$

$$b = 1 - a = \frac{1 - \varphi}{\varphi + 2} = -\frac{1}{\varphi(\varphi + 2)}$$

$$C_h = \frac{1}{\varphi + 2}\left(\varphi^{h+3} + \left(-\frac{1}{\varphi}\right)^{h+1}\right) - 1$$

$$\frac{1}{\varphi + 2} = \frac{2}{5 + \sqrt{5}} = \frac{1}{\sqrt{5}\varphi}$$

$$C_h = \frac{1}{\sqrt{5}}\left(\left(\frac{1 + \sqrt{5}}{2}\right)^{h+2} - \left(\frac{1 - \sqrt{5}}{2}\right)^{h+2}\right) - 1$$

$$B_h = 2^{\frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^{h+2} - \left(\frac{1-\sqrt{5}}{2}\right)^{h+2}\right) - 1}$$

□

ix An implementation of BST and ASL tree in Python

```python
class Node:
    def __init__(self, value=None, parent=None):
        self.value = value
        self.parent = parent
        self.left_child = None
        self.right_child = None
        self.height = 1  # only used in AVL, leaf 1


class BST:
    def __init__(self):
        self.root = None

    # Time complexity: O(height)
    def search(self, x):
        if self.root == None:
            return None
        node = self.root
        while node != None:
            if node.value > x:
                node = node.left_child
            elif node.value < x:
                node = node.right_child
            else:
                return node
        return None

    # Time complexity: O(height)
    def find_min(self):
        if self.root == None:
            return None
        node = self.root
        while node.left_child != None:
            node = node.left_child
        return node.value

```

```
    # Time complexity: O(height)
    def find_max(self):
        if self.root == None:
            return None
        node = self.root
        while node.right_child != None:
            node = node.right_child
        return node.value

    # Time complexity: O(height)
    def upper_bound(self, x):
        r = None
        node = self.root
        while node != None:
            if node.value <= x:
                r = node
                node = node.right_child
            else:
                node = node.left_child
        return r

    # Time complexity: O(height)
    def lower_bound(self, x):
        r = None
        node = self.root
        while node != None:
            if node.value >= x:
                r = node
                node = node.left_child
            else:
                node = node.right_child
        return r

    # Time complexity: O(height)
    def insert(self, x):
        if self.root == None:
            self.root = Node(x)
            return self.root
        node = self.root
        while True:
            if node.value > x:
                if node.left_child == None:
                    node.left_child = Node(x, node)
                    return node.left_child
                node = node.left_child
            elif node.value < x:
                if node.right_child == None:
                    node.right_child = Node(x, node)
                    return node.right_child
                node = node.right_child
            else:
                return None

    def _transplant(self, node, new):
        if node.parent == None:
            self.root = new
        elif node.parent.left_child == node:
            node.parent.left_child = new
        elif node.parent.right_child == node:
            node.parent.right_child = new
        if new != None:
            new.parent = node.parent
```

```
        return node.parent

    # Time complexity: O(height)
    def remove(self, x):
        node = self.search(x)
        if node == None:
            return None
        if node.right_child == None and node.left_child == None:
            return self._transplant(node, None)
        if node.right_child == None:
            return self._transplant(node, node.left_child)
        if node.left_child == None:
            return self._transplant(node, node.right_child)
        succ = node.right_child
        while succ.left_child != None:
            succ = succ.left_child
        node.value = succ.value
        return self._transplant(succ, succ.right_child)

    # Time complexity: O(n)
    def preorder(self):
        result, stack = [], []
        if self.root != None:
            stack.append(self.root)
        while stack != []:
            node = stack.pop()
            result.append(node.value)
            if node.right_child != None:
                stack.append(node.right_child)
            if node.left_child != None:
                stack.append(node.left_child)
        return result

    # Time complexity: O(n)
    def postorder(self):
        result, stack, node = [], [], self.root
        last_visited = None
        while stack != [] or node != None:
            if node != None:
                stack.append(node)
                node = node.left_child
            else:
                right = stack[-1].right_child
                if right != None and last_visited is not right:
                    node = right
                else:
                    result.append(stack[-1].value)
                    last_visited = stack.pop()
        return result

    # Time complexity: O(n)
    def inorder(self):
        result, stack, node = [], [], self.root
        while stack != [] or node != None:
            while node != None:
                stack.append(node)
                node = node.left_child
            node = stack.pop()
            result.append(node.value)
            node = node.right_child
        return result

```

```python

# height = Theta(log n)
class AVL(BST):
    @staticmethod
    def _height(node):
        return node.height if node != None else 0

    @staticmethod
    def _update(node):
        node.height = 1 + max(
            AVL._height(node.left_child), AVL._height(node.right_child)
        )

    @staticmethod
    def _bf(node):
        return AVL._height(node.left_child) - AVL._height(node.right_child)

    def _rotate_right(self, node):
        child = node.left_child
        self._transplant(node, child)
        if child.right_child != None:
            child.right_child.parent = node
        node.left_child = child.right_child
        child.right_child = node
        node.parent = child
        self._update(node)
        self._update(child)
        return child

    def _rotate_left(self, node):
        child = node.right_child
        self._transplant(node, child)
        if child.left_child != None:
            child.left_child.parent = node
        node.right_child = child.left_child
        child.left_child = node
        node.parent = child
        self._update(node)
        self._update(child)
        return child

    def _rebalance(self, node):
        self._update(node)
        bf = self._bf(node)
        if bf > 1:
            child = node.left_child
            if self._bf(child) < 0:
                self._rotate_left(child)
            return self._rotate_right(node)
        if bf < -1:
            child = node.right_child
            if self._bf(child) > 0:
                self._rotate_right(child)
            return self._rotate_left(node)
        if node.parent == None:
            return node
        return self._rebalance(node.parent)

    def insert(self, x):
        node = super().insert(x)
        if node != None and node.parent != None:
            self._rebalance(node.parent)
```

```
223
224     def remove(self, x):
225         node = super().remove(x)
226         while node != None:
227             node = self._rebalance(node).parent
```

VII Splay tree

i Splay tree

A splay tree is a self-adjusting binary search tree (BST) that keeps no info of height and in which every access operation moves the accessed node to the root through an operation called splaying, invented by Daniel Dominic Sleator and Robert Endre Tarjan.

ii Splaying

When a node x is accessed, a splay operation consisting of a sequence of splay steps is performed on x to move it to the root, called splaying x. Let the parent of x be p and the parent of p be g. There are three types of splay steps, each of which has two symmetric variants:

- Zig step / simple rotation: This is done if x is a child of the root. We perform a rotation on the edge between x and root. This only happens as the last step and only happens when x originally has odd depth.
- Zig-zig step: This is done either when x is a left child of p and p is a left child of g, or when x is a right child of p and p is a right child of g. We perform a rotation on the edge between p and g and then a rotation on the edge between x and p.
- Zig-zag step: This is done either when x is a left child of p and p is a right child of g, or when x is a right child of p and p is a left child of g. We perform a rotation on the edge between x and p and then a rotation on the edge between x, now at the original position of p, and g.

iii Searching and finding minimum/maximum

Searching and finding minimum/maximum in splay tree involves first doing the operation as with a normal BST and then splay the found node if found.

iv Join

Given two BSTs S and T such that all elements of S are smaller than all elements of T, the following steps can be used to join them to a single BST: either

1. Splay the largest item in S. Now this item is in the root r of S and has no right child.

2. Set the right child of the new root r of S to the root of T.

or

1. Splay the smallest item in T. Now this item is in the root r of T and has no left child.
2. Set the left child of the new root r of T to the root of S.

v Split

Given a BST and a value x, splitting the BST at x returns two new BSTs, one containing all elements less than or equal to x, L, and the other containing all elements greater than x, R. This can be done with:

1. Search for x in the tree. If we found it, let y be that node; otherwise let y be the node we stop at.
2. Splay y.
3. Split its right subtree, which is R, from the rest of the tree, which is L.

vi Insertion

To insert a value x into a splay tree, there are two ways: either

1. Insert x as with a normal BST.
2. Splay x.

or

1. Split at x. However, if x is found in the tree, stop after splaying x.
2. Create a new tree with x as root and the two split trees as its subtrees.

vii Deletion

To delete a node x from a splay tree: either

1. Search for x.
2. Splay x.
3. Delete x and join the two subtrees of x.

or

1. Delete x as with a normal BST.
2. Splay the original parent of the deleted x.